

# torch.linalg.inv#

<https://pytorch.org/docs/stable/generated/torch.linalg.inv.html#torch.linalg.inv>

## Description:

Computes the inverse of a square matrix if it exists. Throws a `RuntimeError` if the matrix is not invertible. Letting  $K \in \{\mathbb{R}, \mathbb{C}\}$ , for a matrix  $A \in K^{n \times n}$ , its inverse matrix  $A^{-1} \in K^{n \times n}$  (if it exists) is defined as where  $I_n$  is the  $n$ -dimensional identity matrix. The inverse matrix exists if and only if  $A$  is invertible. In this case, the inverse is unique. Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if  $A$  is a batch of matrices then the output has the same batch dimensions.

## Function Signature

---

```
torch.linalg.inv(A, *, out=None) → Tensor#
```

Parameters

Keyword Arguments

Raises

## Parameters

---

### Keyword

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

## Code Examples

---

```
linalg.solve(A, B) == linalg.inv(A) @ B # When B is a matrix
```

```
>>> A = torch.randn(4, 4)
>>> Ainv = torch.linalg.inv(A)
>>> torch.dist(A @ Ainv, torch.eye(4))
tensor(1.1921e-07)
```

```
>>> A = torch.randn(2, 3, 4, 4) # Batch of matrices
>>> Ainv = torch.linalg.inv(A)
>>> torch.dist(A @ Ainv, torch.eye(4))
tensor(1.9073e-06)
```

```
>>> A = torch.randn(4, 4, dtype=torch.complex128) # Complex
matrix
>>> Ainv = torch.linalg.inv(A)
>>> torch.dist(A @ Ainv, torch.eye(4))
tensor(7.5107e-16, dtype=torch.float64)
```

## Notes & Warnings

---

### NOTE

Note When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see `torch.linalg.inv_ex()`.

### NOTE

Note Consider using `torch.linalg.solve()` if possible for multiplying a matrix on the left by the inverse, as: `linalg.solve(A, B) == linalg.inv(A) @ B` # When B is a matrix It is always preferred to use `solve()` when possible, as it is faster and more numerically stable than computing the inverse explicitly.

### NOTE

See also `torch.linalg.pinv()` computes the pseudoinverse (Moore-Penrose inverse) of matrices of any shape. `torch.linalg.solve()` computes `A.inv() @ B` with a numerically stable algorithm.

## Detailed Documentation

---

### Documentation

Computes the inverse of a square matrix if it exists. Throws a `RuntimeError` if the matrix is not invertible.

Letting  $K \in \{\mathbb{R}, \mathbb{C}\}$ , for a matrix  $A \in K^{n \times n}$

$\mathbb{K}^{n \times n}$   $A \in \mathbb{K}^{n \times n}$ , its inverse matrix  $A^{-1} \in \mathbb{K}^{n \times n}$   $A^{-1} \in \mathbb{K}^{n \times n}$  (if it exists) is defined as

where  $I_n$  is the  $n$ -dimensional identity matrix.

The inverse matrix exists if and only if  $A$  is invertible. In this case, the inverse is unique.

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if  $A$  is a batch of matrices then the output has the same batch dimensions.

When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see `torch.linalg.inv_ex()`.

Consider using `torch.linalg.solve()` if possible for multiplying a matrix on the left by the inverse, as:

It is always preferred to use `solve()` when possible, as it is faster and more numerically stable than computing the inverse explicitly.

`torch.linalg.pinv()` computes the pseudoinverse (Moore-Penrose inverse) of matrices of any shape.

`torch.linalg.solve()` computes  $A^{-1} @ B$  with a numerically stable algorithm.

$A$  (Tensor) – tensor of shape  $(*, n, n)$  where  $*$  is zero or more batch dimensions consisting of invertible matrices.

$out$  (Tensor, optional) – output tensor. Ignored if None. Default: None.

**RuntimeError** – if the matrix  $A$  or any matrix in the batch of matrices  $A$  is not invertible.

